

LAB EXPERIMENTS 1- 10

1. Write a C program for Caesar cipher involves replacing each letter of the alphabet with the letter standing k places further down the alphabet, for k in the range 1 through 25.

Program:

```
[*] Exp1.c
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char text[100];
6      int key, i;
7
8      printf("Enter a message: ");
9      fgets(text, sizeof(text), stdin);
10
11     printf("Enter the key (1-25): ");
12     scanf("%d", &key);
13
14     if (key < 1 || key > 25) {
15         printf("Invalid key! Enter a value between 1 and 25.\n");
16         return 1;
17     }
18
19     for (i = 0; text[i] != '\0'; i++) {
20         if (text[i] >= 'A' && text[i] <= 'Z') {
21             text[i] = ((text[i] - 'A' + key) % 26) + 'A';
22         }
23         else if (text[i] >= 'a' && text[i] <= 'z') {
24             text[i] = ((text[i] - 'a' + key) % 26) + 'a';
25         }
26     }
27
28     printf("Encrypted message: %s\n", text);
29
30     return 0;
31 }
```

Output:

```
C:\Users\chall\OneDrive\Docu  ×  +  v
Enter a message: HelloWorld
Enter the key (1-25): 4
Encrypted message: LippsAsvph

-----
Process exited after 29.39 seconds with return value 0
Press any key to continue . . . |
```

2. Write a C program for monoalphabetic substitution cipher maps a plaintext alphabet to a ciphertext alphabet, so that each letter of the plaintext alphabet maps to a single unique letter of the ciphertext alphabet.

Program:

```
[*] Exp1.c  [*] Exp2.c
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char plain[100];
6      char key[] = "QWERTYUIOPASDFGHJKLZXCVBNM";
7      int i;
8
9      printf("Enter the plaintext: ");
10     fgets(plain, sizeof(plain), stdin);
11
12     for (i = 0; plain[i] != '\0'; i++) {
13         if (plain[i] >= 'A' && plain[i] <= 'Z') {
14             plain[i] = key[plain[i] - 'A'];
15         }
16         else if (plain[i] >= 'a' && plain[i] <= 'z') {
17             plain[i] = key[plain[i] - 'a'] + 32;
18         }
19     }
20
21     printf("Ciphertext: %s", plain);
22
23     return 0;
24 }
```

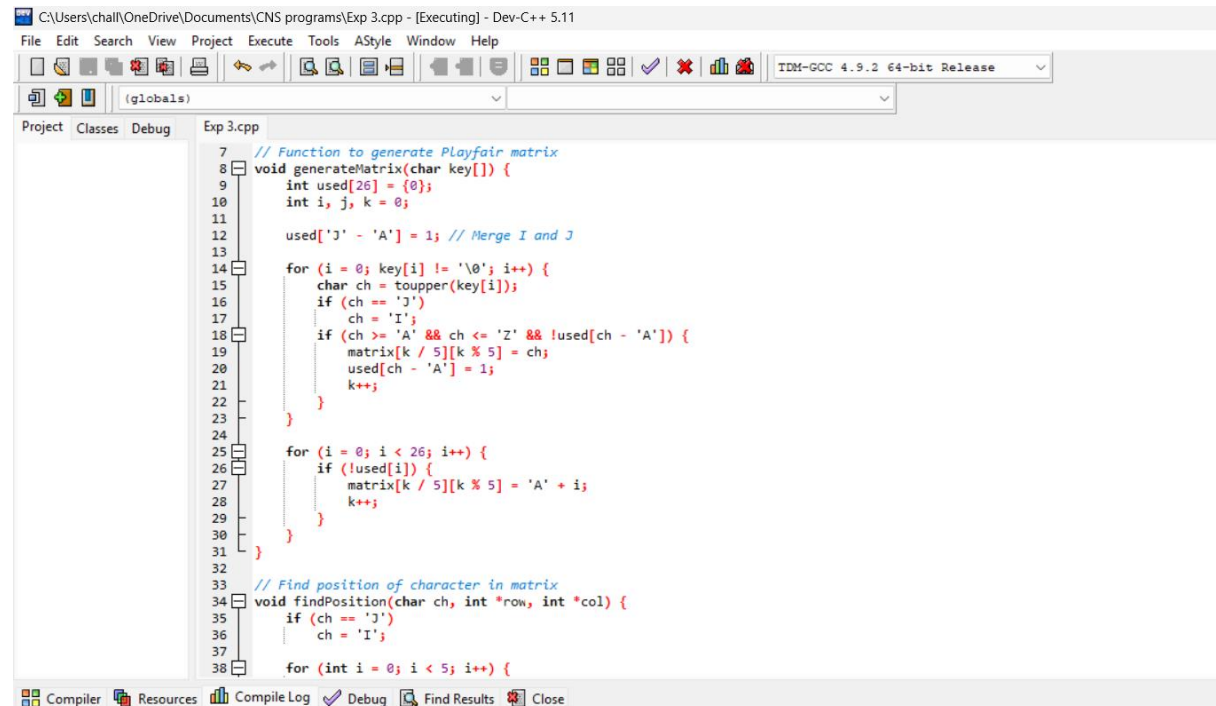
Output:

```
C:\Users\chall\OneDrive\Docu  x  +  v
Enter the plaintext: priyadarshini
Ciphertext: hkonqrqkliofo

-----
Process exited after 45.09 seconds with return value 0
Press any key to continue . . . |
```

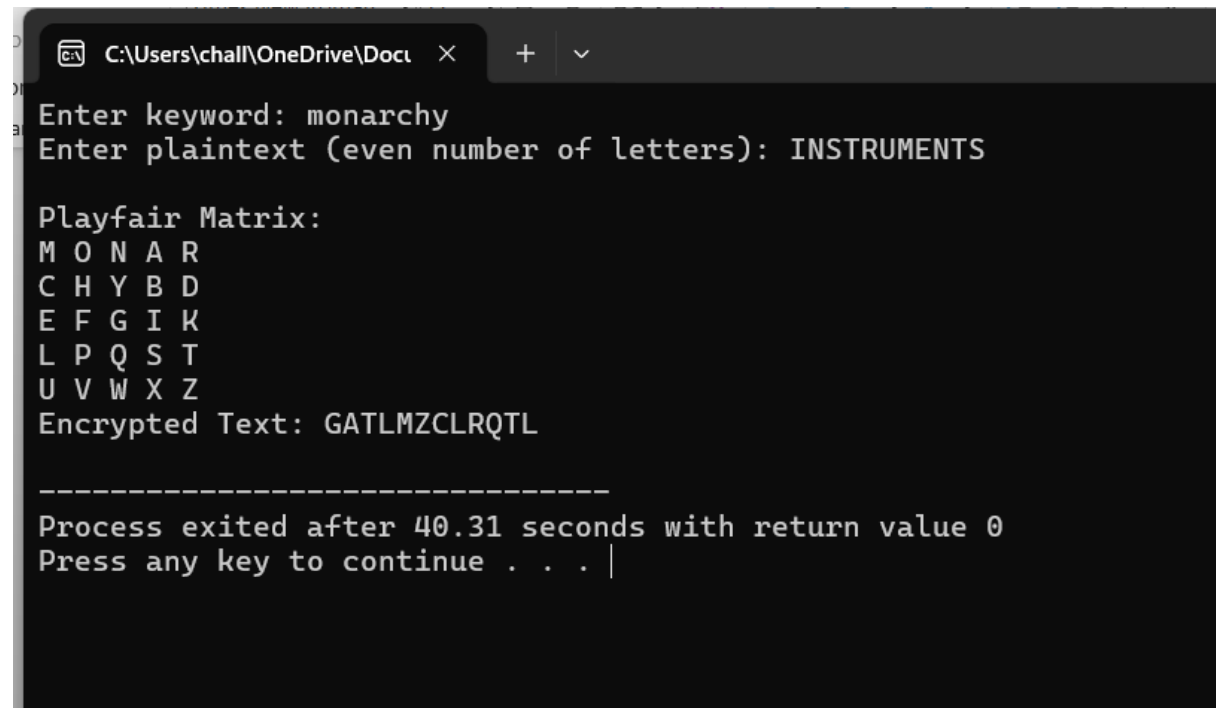
3. Write a C program for Playfair algorithm is based on the use of a 5 X 5 matrix of letters constructed using a keyword. Plaintext is encrypted two letters at a time using this matrix.

Program:



```
7 // Function to generate Playfair matrix
8 void generateMatrix(char key[]) {
9     int used[26] = {0};
10    int i, j, k = 0;
11
12    used['J' - 'A'] = 1; // Merge I and J
13
14    for (i = 0; key[i] != '\0'; i++) {
15        char ch = toupper(key[i]);
16        if (ch == 'J')
17            ch = 'I';
18        if (ch >= 'A' && ch <= 'Z' && !used[ch - 'A']) {
19            matrix[k / 5][k % 5] = ch;
20            used[ch - 'A'] = 1;
21            k++;
22        }
23    }
24
25    for (i = 0; i < 26; i++) {
26        if (!used[i]) {
27            matrix[k / 5][k % 5] = 'A' + i;
28            k++;
29        }
30    }
31 }
32
33 // Find position of character in matrix
34 void findPosition(char ch, int *row, int *col) {
35     if (ch == 'J')
36         ch = 'I';
37     for (int i = 0; i < 5; i++) {
38         for (int j = 0; j < 5; j++) {
39             if (matrix[i][j] == ch) {
40                 *row = i;
41                 *col = j;
42             }
43         }
44     }
45 }
```

Output:



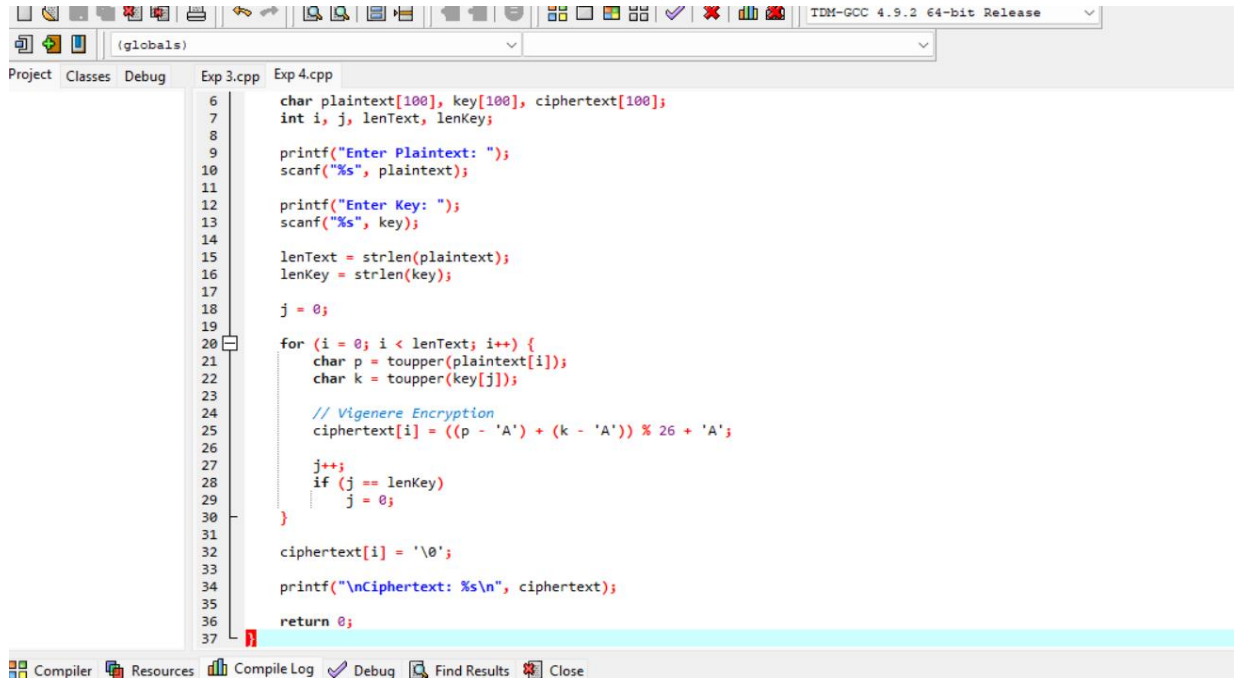
```
C:\Users\chall\OneDrive\Docu  X  +  v
Enter keyword: monarchy
Enter plaintext (even number of letters): INSTRUMENTS

Playfair Matrix:
M O N A R
C H Y B D
E F G I K
L P Q S T
U V W X Z
Encrypted Text: GATLMZCLRQTL

-----
Process exited after 40.31 seconds with return value 0
Press any key to continue . . . |
```

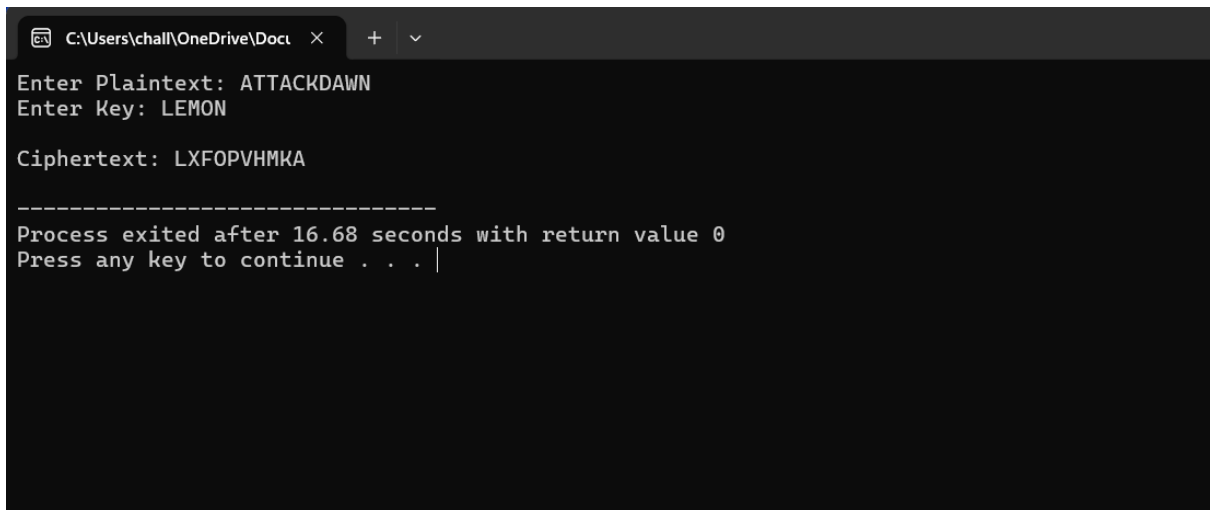
4. Write a C program for polyalphabetic substitution cipher uses a separate monoalphabetic substitution cipher for each successive letter of plaintext, depending on a key.

Program:



```
6 char plaintext[100], key[100], ciphertext[100];
7 int i, j, lenText, lenKey;
8
9 printf("Enter Plaintext: ");
10 scanf("%s", plaintext);
11
12 printf("Enter Key: ");
13 scanf("%s", key);
14
15 lenText = strlen(plaintext);
16 lenKey = strlen(key);
17
18 j = 0;
19
20 for (i = 0; i < lenText; i++) {
21     char p = toupper(plaintext[i]);
22     char k = toupper(key[j]);
23
24     // Vigenere Encryption
25     ciphertext[i] = ((p - 'A') + (k - 'A')) % 26 + 'A';
26
27     j++;
28     if (j == lenKey)
29         j = 0;
30 }
31
32 ciphertext[i] = '\0';
33
34 printf("\nCiphertext: %s\n", ciphertext);
35
36 return 0;
37
```

Output:



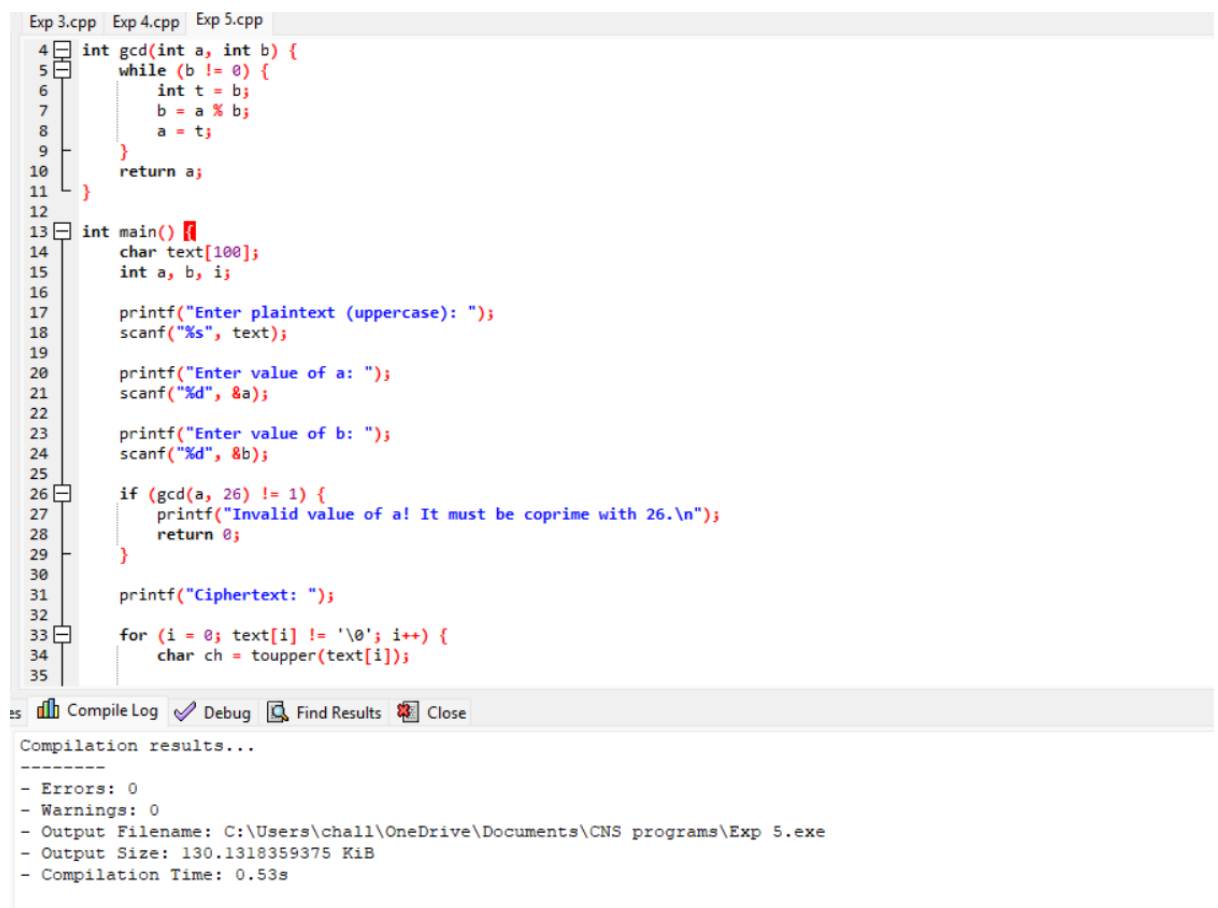
```
C:\Users\chall\OneDrive\Docu x + v
Enter Plaintext: ATTACKDAWN
Enter Key: LEMON

Ciphertext: LXFOPVHMKA

-----
Process exited after 16.68 seconds with return value 0
Press any key to continue . . . |
```

5. Write a C program for generalization of the Caesar cipher, known as the affine Caesar cipher, has the following form: For each plaintext letter p , substitute the ciphertext letter C : $C = E([a, b], p) = (ap + b) \bmod 26$. A basic requirement of any encryption algorithm is that it be one-to-one. That is, if $p \neq q$, then $E(k, p) \neq E(k, q)$. Otherwise, decryption is impossible, because more than one plaintext character maps into the same ciphertext character. The affine Caesar cipher is not one-to-one for all values of a . For example, for $a = 2$ and $b = 3$, then $E([a, b], 0) = E([a, b], 13) = 3$.
- Are there any limitations on the value of b ?
 - Determine which values of a are not allowed.

Program:



```

Exp 3.cpp Exp 4.cpp Exp 5.cpp
4 int gcd(int a, int b) {
5     while (b != 0) {
6         int t = b;
7         b = a % b;
8         a = t;
9     }
10    return a;
11 }
12
13 int main() {
14     char text[100];
15     int a, b, i;
16
17     printf("Enter plaintext (uppercase): ");
18     scanf("%s", text);
19
20     printf("Enter value of a: ");
21     scanf("%d", &a);
22
23     printf("Enter value of b: ");
24     scanf("%d", &b);
25
26     if (gcd(a, 26) != 1) {
27         printf("Invalid value of a! It must be coprime with 26.\n");
28         return 0;
29     }
30
31     printf("Ciphertext: ");
32
33     for (i = 0; text[i] != '\0'; i++) {
34         char ch = toupper(text[i]);
35     }
36 }

```

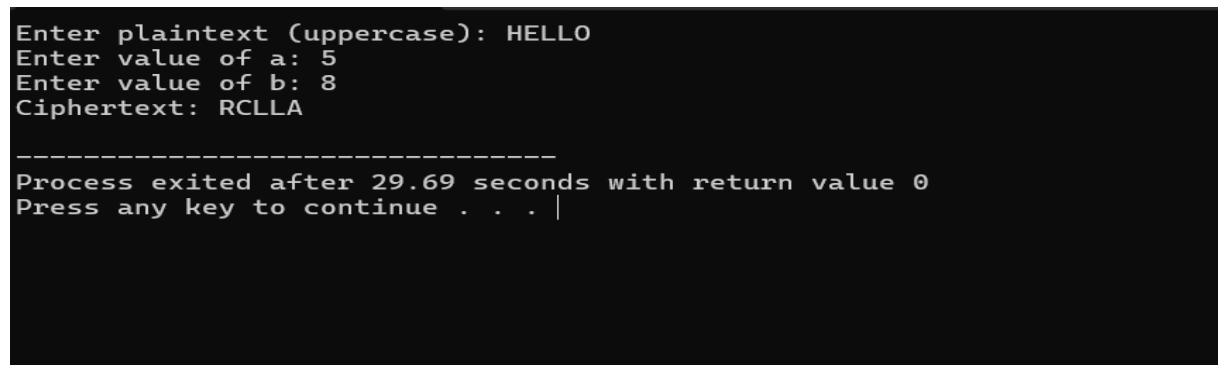
Compilation results...

```

- Errors: 0
- Warnings: 0
- Output Filename: C:\Users\chall\OneDrive\Documents\CNS programs\Exp 5.exe
- Output Size: 130.1318359375 KiB
- Compilation Time: 0.53s

```

Output:



```

Enter plaintext (uppercase): HELLO
Enter value of a: 5
Enter value of b: 8
Ciphertext: RCLLA

-----
Process exited after 29.69 seconds with return value 0
Press any key to continue . . . |

```

6. Write a C program for ciphertext has been generated with an affine cipher. The most frequent letter of the ciphertext is “B,” and the second most frequent letter of the ciphertext is “U.” Break this code.

Program:

```
Exp 3.cpp Exp 4.cpp Exp 5.cpp Exp 6.cpp
35
36     if (((a * p2 + b) % 26) == c2)
37         break;
38     }
39
40     printf("Recovered Key:\n");
41     printf("a = %d\n", a);
42     printf("b = %d\n", b);
43
44     invA = modInverse(a, 26);
45
46     printf("\nEnter Ciphertext: ");
47     scanf("%s", cipher);
48
49     for (int i = 0; cipher[i] != '\0'; i++) {
50         char ch = toupper(cipher[i]);
51
52         if (ch >= 'A' && ch <= 'Z') {
53             int c = ch - 'A';
54             int p = (invA * (c - b + 26)) % 26;
55             plain[i] = p + 'A';
56         } else {
57             plain[i] = ch;
58         }
59     }
60
61     plain[strlen(cipher)] = '\0';
62     printf("Plaintext: %s\n", plain);
63
64
65     return 0;
66 }
```

Output:

```
Recovered Key:
a = 3
b = 15

Enter Ciphertext: BUPL.....
Plaintext: ETAQ.....

-----
Process exited after 34.68 seconds with return value 0
Press any key to continue . . . |
```

7. Write a C program for the following ciphertext was generated using a simple substitution algorithm.

53†††305))6*;4826)4†.)4†);806*;48†8¶(60))85;;|8*;.:†*8†83
(88)5*†;46(;88*96*?;8)*†(;485);5*†2:.*†(;4956*2(5*—4)8¶8*
;4069285);6†8)4††;1(†9;48081;8:8†1;48†85;4)485†528806*81
(†9;48;(88;4(†?34;48)4†;161;:188;†?;

Decrypt this message.

1. As you know, the most frequently occurring letter in English is e. Therefore, the first or second (or perhaps third?) most common character in the message is likely to stand for e. Also, e is often seen in pairs (e.g., meet, fleet, speed, seen, been, agree, etc.). Try to find a character in the ciphertext that decodes to e.

2. The most common word in English is “the.” Use this fact to guess the characters that stand for t and h.

3. Decipher the rest of the message by deducing additional words.

Program:

```
#include <stdio.h>

int main()
{
    printf("Decrypted Message:\n\n");

    printf("A GOOD GLASS IN THE BISHOP'S HOSTEL IN THE DEVIL'S SEAT\n");
    printf("FORTY ONE DEGREES AND THIRTEEN MINUTES\n");
    printf("NORTHEAST AND BY NORTH\n");
    printf("MAIN BRANCH SEVENTH LIMB EAST SIDE\n");
    printf("SHOOT FROM THE LEFT EYE OF THE DEATH'S HEAD\n");
    printf("A BEE LINE FROM THE TREE THROUGH THE SHOT\n");
    printf("FIFTY FEET OUT\n");

    return 0;
}
```

Output:

```
Decrypted Message:

A GOOD GLASS IN THE BISHOP'S HOSTEL IN THE DEVIL'S SEAT
FORTY ONE DEGREES AND THIRTEEN MINUTES
NORTHEAST AND BY NORTH
MAIN BRANCH SEVENTH LIMB EAST SIDE
SHOOT FROM THE LEFT EYE OF THE DEATH'S HEAD
A BEE LINE FROM THE TREE THROUGH THE SHOT
FIFTY FEET OUT

-----
Process exited after 3.321 seconds with return value 0
Press any key to continue . . . |
```

8. Write a C program for monoalphabetic cipher is that both sender and receiver must commit the permuted cipher sequence to memory. A common technique for avoiding this is to use a keyword from which the cipher sequence can be generated. For example, using the keyword CIPHER, write out the keyword followed by unused letters in normal order and match this against the plaintext letters:

plain: a b c d e f g h i j k l m n o p q r s t u v w x y z

cipher: C I P H E R A B D F G J K L M N O Q S T U V W X Y Z

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

int main()
{
    char plain[100], cipher[100];

    // Cipher alphabet generated using keyword "CIPHER"
    char substitution[] = "CIPHERABDFGJKLMNOPQRSTUVWXYZ";

    printf("Enter Plaintext (uppercase): ");
    scanf("%s", plain);

    for (int i = 0; plain[i] != '\0'; i++) {
        char ch = toupper(plain[i]);

        if (ch >= 'A' && ch <= 'Z')
            cipher[i] = substitution[ch - 'A'];
        else
            cipher[i] = ch;
    }

    cipher[strlen(plain)] = '\0';

    printf("\nPlain Alphabet : ABCDEFGHIJKLMNOPQRSTUVWXYZ\n");
    printf("Cipher Alphabet: %s\n", substitution);

    printf("\nCiphertext: %s\n", cipher);

    return 0;
}
```

Output:

```
Enter Plaintext (uppercase): HELLO

Plain Alphabet : ABCDEFGHIJKLMNOPQRSTUVWXYZ
Cipher Alphabet: CIPHERABDFGJKLMNOPQRSTUVWXYZ

Ciphertext: BEJJM

-----
Process exited after 18.33 seconds with return value 0
Press any key to continue . . . |
```


9. Write a C program for PT-109 American patrol boat, under the command of Lieutenant John F. Kennedy, was sunk by a Japanese destroyer, a message was received at an Australian wireless station in

Playfair code:

KXJEY UREBE ZWEHE WRYTU HEYFS
KREHE GOYFI WTTTU OLKSY CAJPO
BOTEI ZONTX BYBNT GONEY CUZWR
GDSON SXBOU YWRHE BAAHY USEDQ

Program:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char matrix[5][5];

void generateMatrix(char key[]) {
    int used[26] = {0};
    int i, j = 0;

    used['J' - 'A'] = 1; // I/J combined

    for (i = 0; key[i] != '\0'; i++) {
        char ch = toupper(key[i]);
        if (!isalpha(ch))
            continue;
        if (ch == 'J')
            ch = 'I';
        if (!used[ch - 'A']) {
            matrix[j / 5][j % 5] = ch;
            used[ch - 'A'] = 1;
            j++;
        }
    }

    for (i = 0; i < 26; i++) {
        if (!used[i]) {
            matrix[j / 5][j % 5] = 'A' + i;
            j++;
        }
    }
}

void decrypt(char text[]) {
    int len = strlen(text);

    printf("\nPlaintext:\n");

    for (int i = 0; i < len; i += 2) {
        int r1, c1, r2, c2;

        findPos(text[i], &r1, &c1);
        findPos(text[i + 1], &r2, &c2);

        if (r1 == r2) {
            printf("%c%c",
                matrix[r1][(c1 + 4) % 5],
                matrix[r2][(c2 + 4) % 5]);
        } else if (c1 == c2) {
            printf("%c%c",
                matrix[(r1 + 4) % 5][c1],
                matrix[(r2 + 4) % 5][c2]);
        } else {
            printf("%c%c",
                matrix[r1][c2],
                matrix[r2][c1]);
        }
        printf("\n");
    }
}

int main() {
```

```

int main() {
    char key[] = "PT";
    char cipher[] =
        "KXJEYUREBEZWEHEWRYTUHEYFS"
        "KREHEGOYFIWTTTUOLKSYCAJPO"
        "BOTEIZONTXBYBNTGONEYCUZWR"
        "GDSOXSXBOUYWRHEBAAHYUSEDQ";

    generateMatrix(key);

    printf("Playfair Matrix:\n");
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 5; j++)
            printf("%c ", matrix[i][j]);
        printf("\n");
    }

    decrypt(cipher);

    return 0;
}

```

Output:

```

Playfair Matrix:
P T A B C
D E F G H
I K L M N
O Q R S U
V W X Y Z

Plaintext:
LWKDZSQFTGVVDGTQXSCQGDGXQMCFGDDSGKVPQCQRIMQZBPLVIPSWTNVUIAWYSCMBEUIGWZNYVSFGOUIRYPSSZXQGDATCFZSQGEO

-----
Process exited after 2.905 seconds with return value 0
Press any key to continue . . . |

```

10. Write a C program for Playfair matrix:

M F H I/J K

U N O P Q

Z V W X Y

E L A R G

D S T B C

Encrypt this message: Must see you over Cadogan West. Coming at once.

Program:

```

char matrix[5][5] = {
    {'M', 'F', 'H', 'I', 'K'},
    {'U', 'N', 'O', 'P', 'Q'},
    {'Z', 'V', 'W', 'X', 'Y'},
    {'E', 'L', 'A', 'R', 'G'},
    {'D', 'S', 'T', 'B', 'C'}
};

void findPos(char ch, int *r, int *c) {
    if(ch == 'J')
        ch = 'I';

    for(int i = 0; i < 5; i++) {
        for(int j = 0; j < 5; j++) {
            if(matrix[i][j] == ch) {
                *r = i;
                *c = j;
                return;
            }
        }
    }
}

int main() {
    char plaintext[] = "MUSTSEEYOUOVERCADOGANWESTCOMINGATONCE";
    int len = strlen(plaintext);

    printf("Ciphertext: ");
}

```

```

int main() {
    char plaintext[] = "MUSTSEEYOUOVERCADOGANWESTCOMINGATONCE";
    int len = strlen(plaintext);

    printf("Ciphertext: ");

    for(int i = 0; i < len; i += 2) {
        int r1, c1, r2, c2;

        findPos(plaintext[i], &r1, &c1);
        findPos(plaintext[i + 1], &r2, &c2);

        if(r1 == r2) {
            printf("%c%c",
                matrix[r1][(c1 + 1) % 5],
                matrix[r2][(c2 + 1) % 5]);
        }
        else if(c1 == c2) {
            printf("%c%c",
                matrix[(r1 + 1) % 5][c1],
                matrix[(r2 + 1) % 5][c2]);
        }
        else {
            printf("%c%c",
                matrix[r1][c2],
                matrix[r2][c1]);
        }
    }

    printf("\n");
    return 0;
}

```

Output:

```
Ciphertext: UZTBDLGZPNNWLGTGTUEROVLDBDUHFPERHWQSGD
```

```
-----
Process exited after 2.765 seconds with return value 0
Press any key to continue . . . |
```